

CSE111

Data Structures

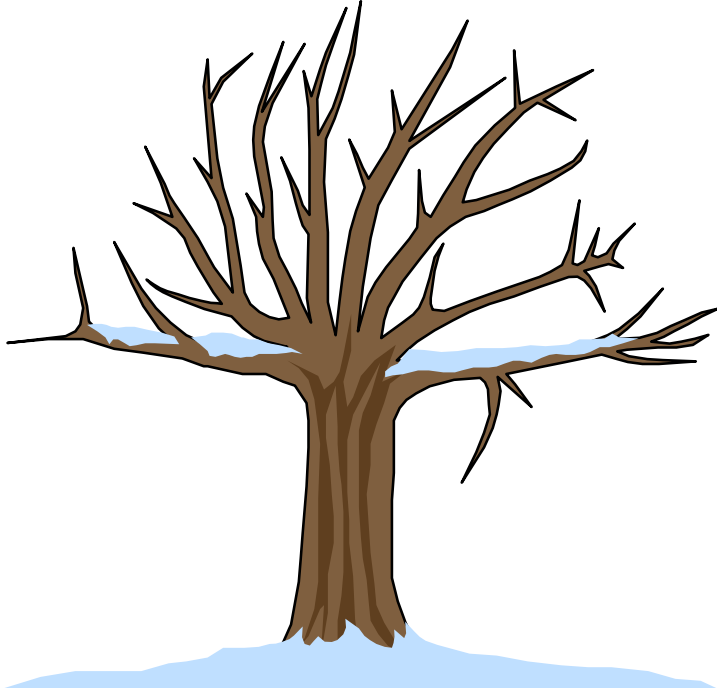
Lecture 04: Trees

Dr Mervat Mikhail

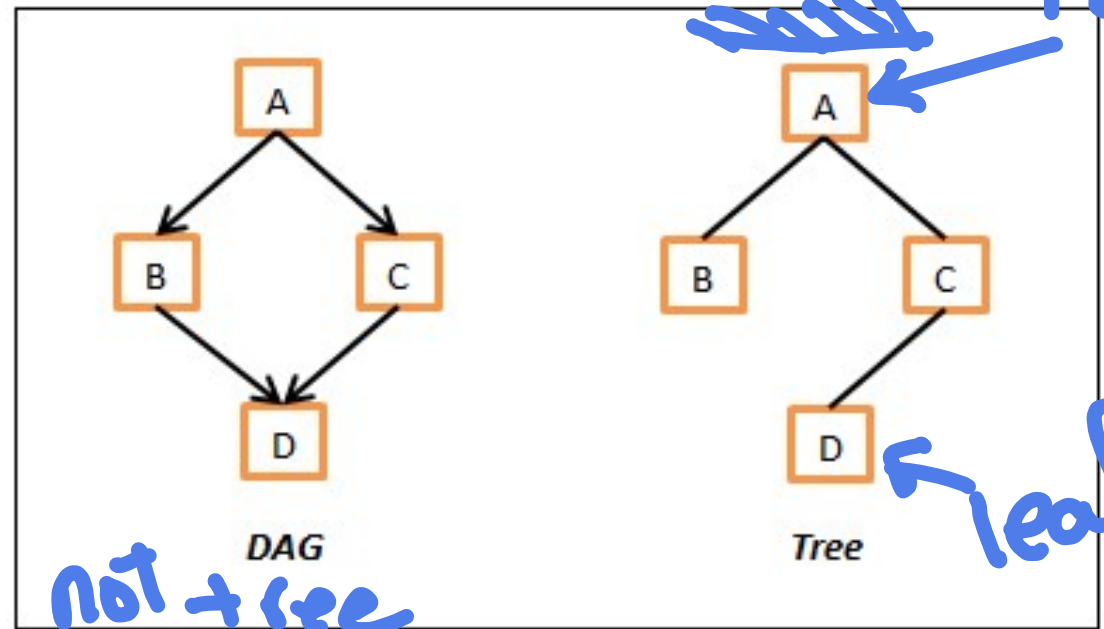
How should I decide which data structure to use ?

- What need to be stored
- Cost of operation
- Memory usage
- Ease of implementation

Trees

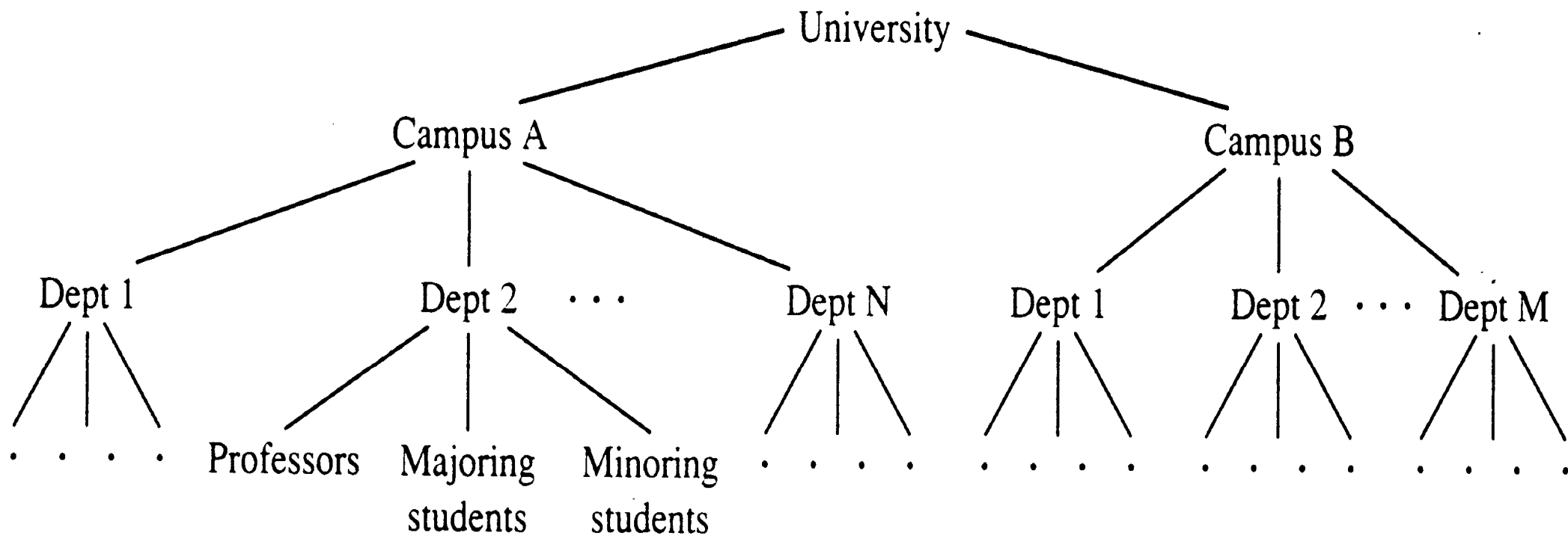


- The **Tree** data type consists of nodes and arcs. Unlike natural trees, these trees are depicted upside down with the root at the top and the leaves at the bottom
- A **Direct Acyclic Graph (DAG)** is a special case of graphs, with no cycles
- Trees are special case of DAG with the idea of parent and children



Trees

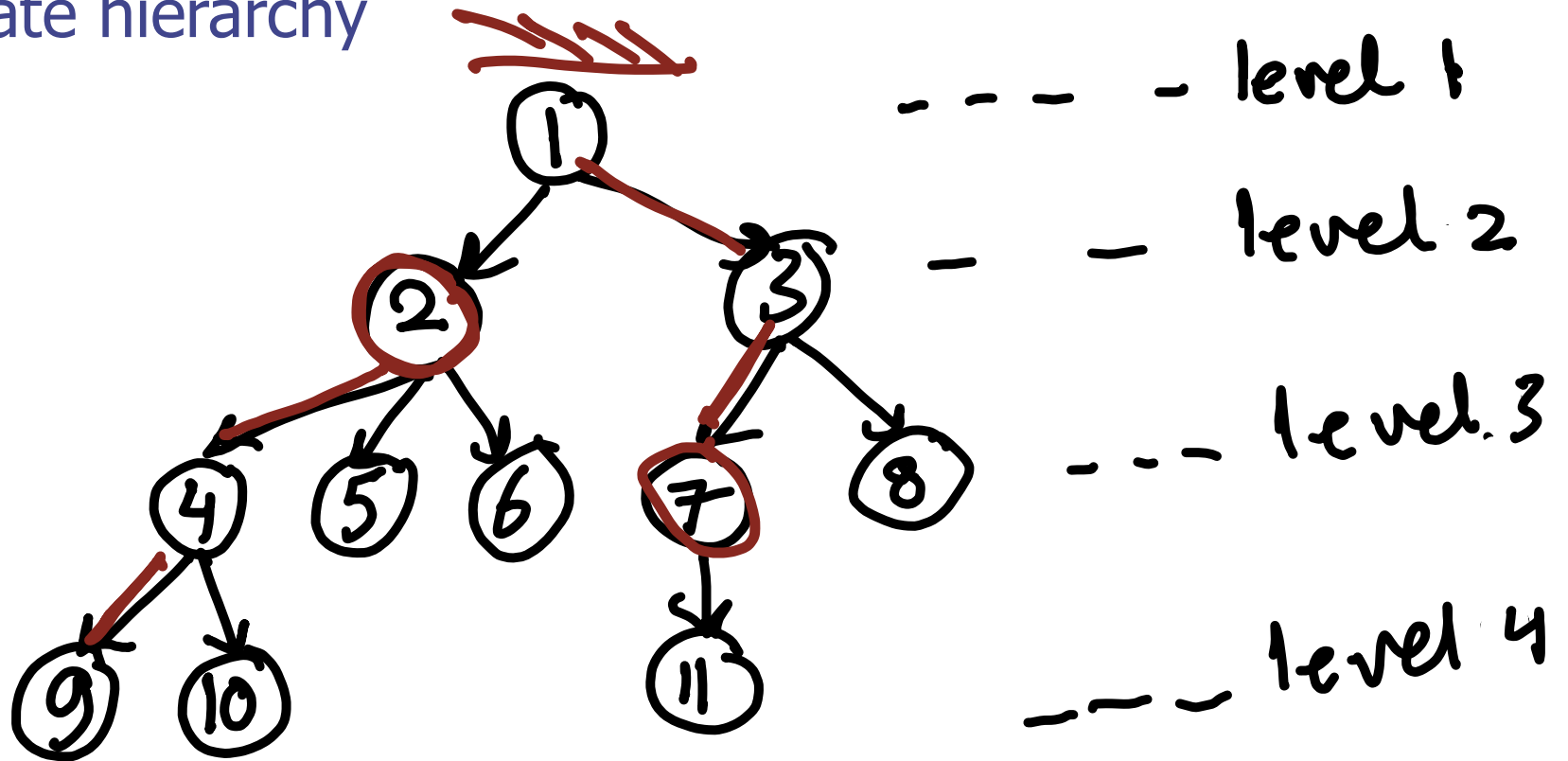
Example: hierarchy of a university



Hierarchical structure of a university shown as a tree.

Tree

Is a collection of entities called nodes linked together to simulate hierarchy



$$k=4, h=3$$

$$h=3$$

Trees

The root is a node that has no **parent**; it can have only child nodes. Leaves, on the other hand, have **no children**. Nodes which are not leaves are called internal nodes. Nodes with the same parent are called siblings.

The depth of a node is the number of edges from the root to the node.

The height of a node is the number of edges from the node to the deepest leaf.

The height of a tree is a height of the root.

$$\text{depth}(7) = 2$$

$$\text{height}(2) = 2$$

$$\text{depth}(5) = 2$$

$$\text{height}(11) = 0$$

$$\text{height}(\text{root}) = 3 = h$$

$$\text{No. of levels} = K, \text{ height} = h$$

$$K = h + 1 \rightarrow \textcircled{1}$$

If No. of nodes = n

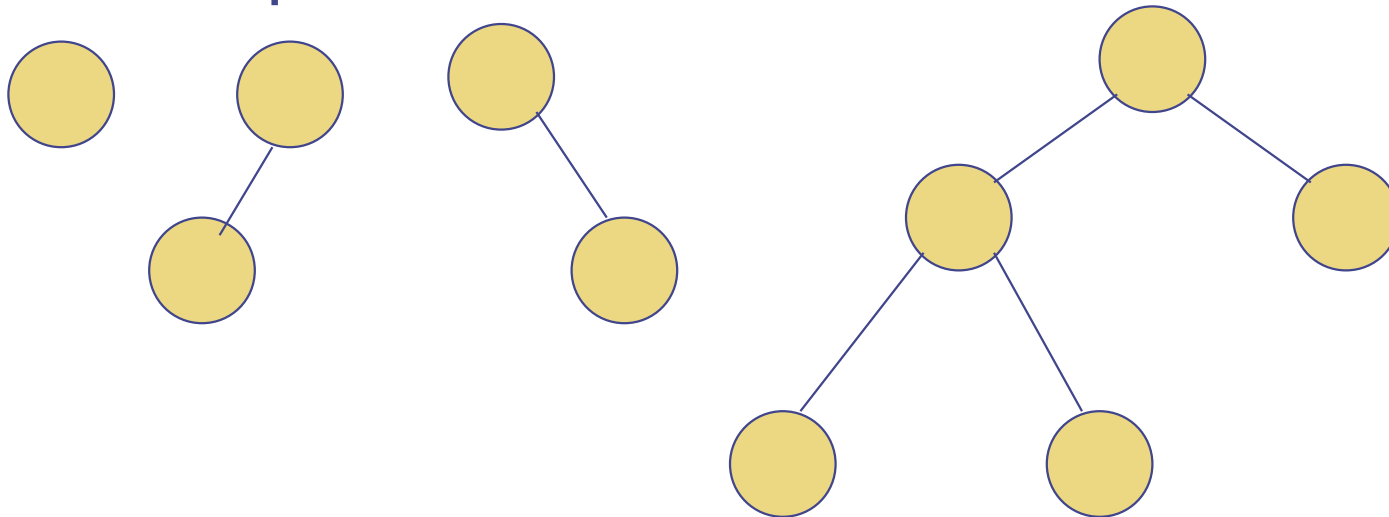
$\therefore$ No. of edges = $n-1$

Binary Trees

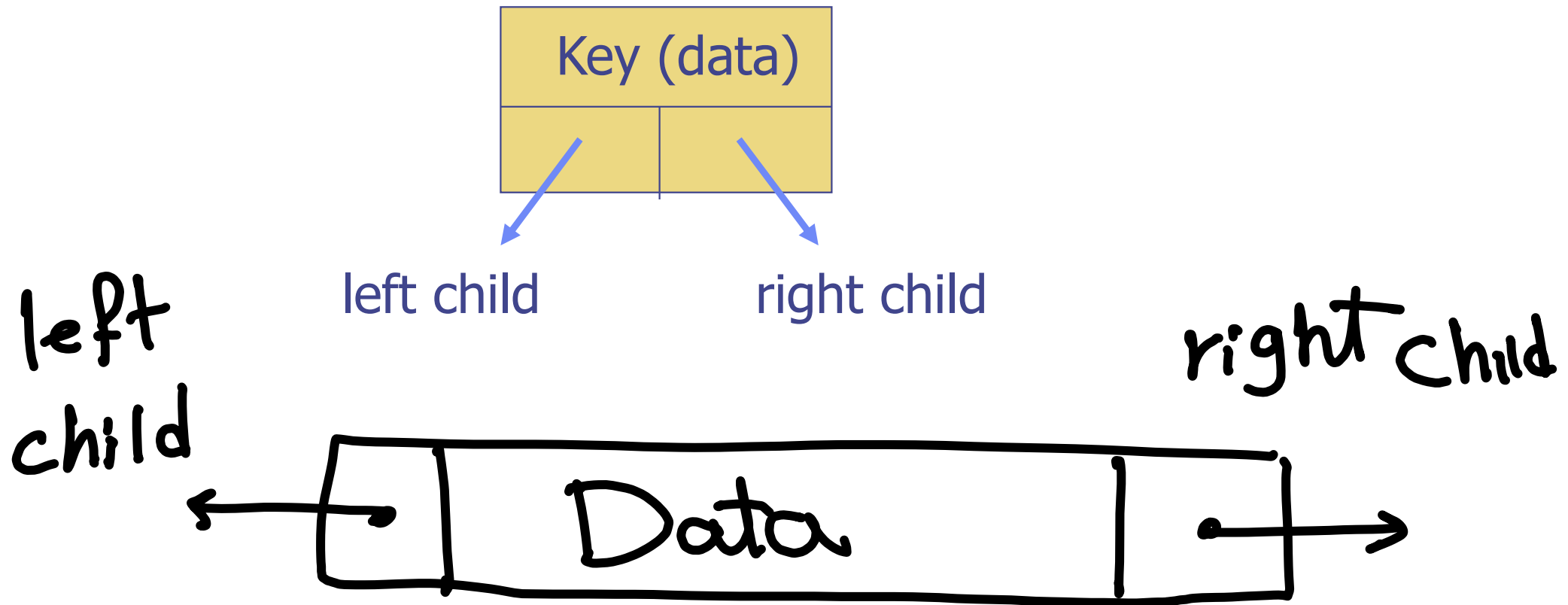
$\{0, 1, 2\}$
↓

A binary tree is a tree whose nodes have at most two children (possibly empty) and each child is designated as either a left child or a right child.

Examples:



Dynamic linked Representation



Applications

- Storing hierarchical data
 - e.g. file system
- Organizing data
- Network routing algorithms
- Efficient search as in BST

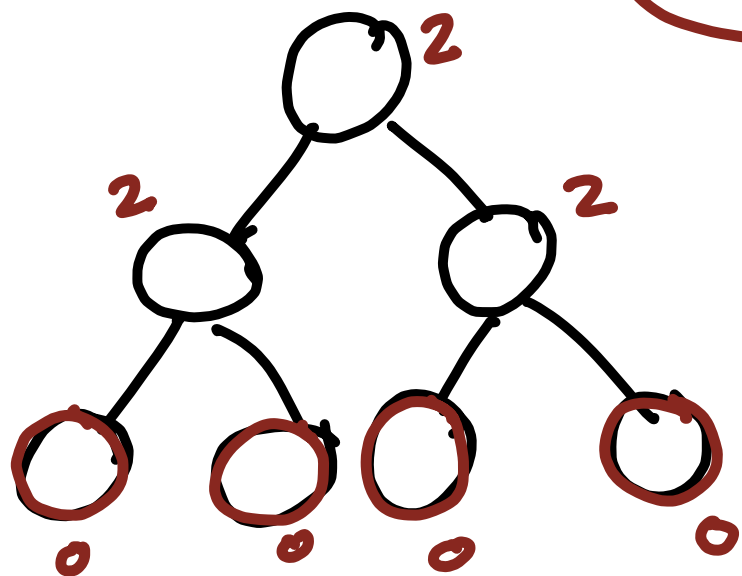
Binary Trees

- A **binary tree** is made of nodes, where each node contains a "left" reference, a "right" reference, and a data element.
- Each node can be connected to at most two nodes, called children.
- A **full (strict) binary tree** is a binary tree in which each node has exactly zero or two children.
- A **complete binary tree** is a binary tree, which is completely filled, such that all the leaves are at the same last level.

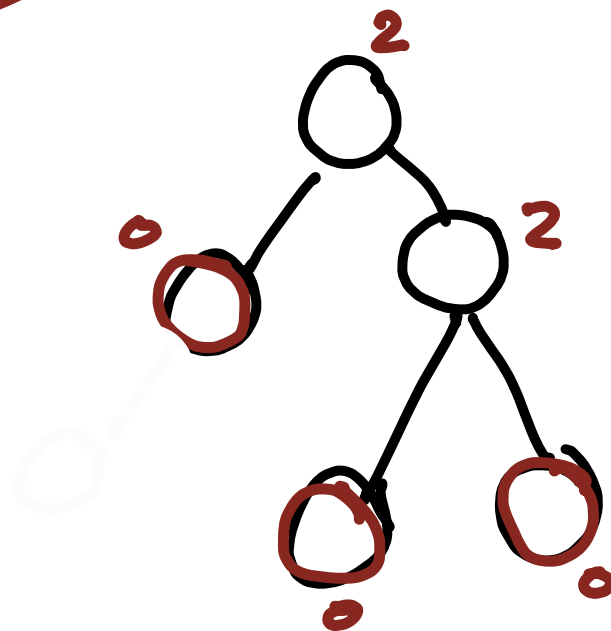
full



Comp full



Full ✓
⇒ Complete ✓



Full ✓
Complete ✗

For Complete binary tree

If k : no. of levels

$$a + ar + ar^2 + \dots + ar^{k-1} = a \frac{1-r^k}{1-r}$$

Total no. of nodes =

$$2^0 + 2^1 + 2^2 + \dots + 2^{k-1}$$

Geom increasing Series with
base = 2, 1st term = 1, No. of terms = k

$$\text{No. of nodes} = \frac{1-2^k}{1-2} = 2^k - 1$$

$\therefore$ Total No. of nodes in a Complete binary tree

$$N = 2^K - 1$$

$$N+1 = 2^K$$

$$K = h+1$$

$$h+1 = \log_2(N+1)$$

$$h = \log_2(N+1) - 1$$

$$K = \log_2(N+1) = O(\log_2 N)$$

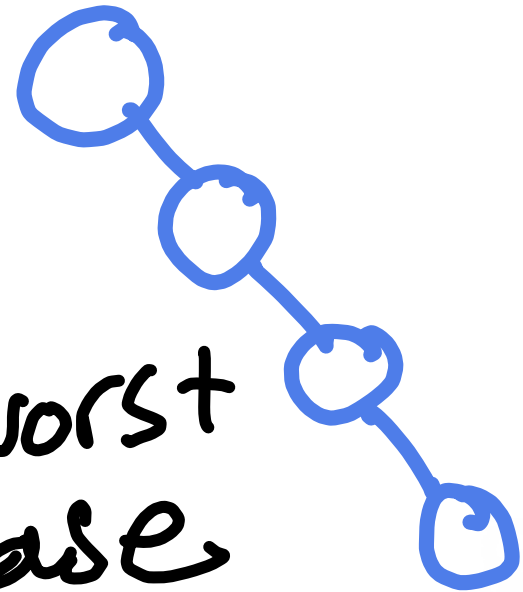
Time Complexity in Comp BT
in terms of height

$$h = O(\log_2 N)$$

If space tree
 $h = N - 1$

$$h = O(N)$$

worst
case



Binary tree traversal

The process of visiting each node in the tree exactly once in some order

BFT

1. Breadth First Traversal – Level order
2. Depth First Traversal – Preorder \Inorder\Postorder

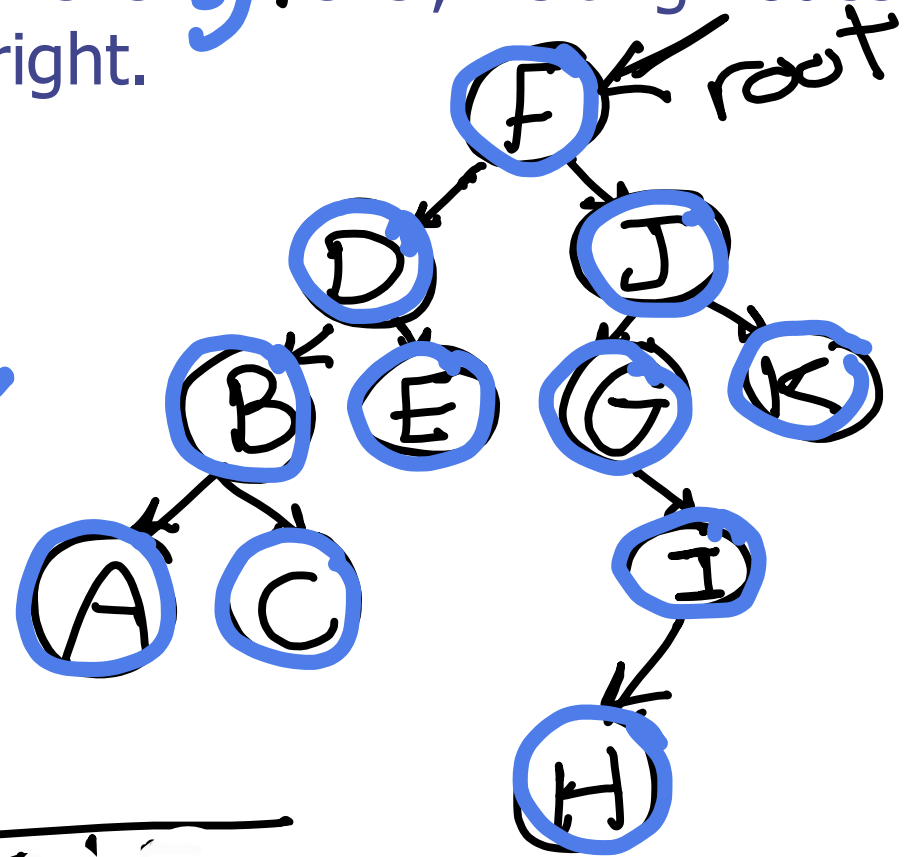
DFT

← — —

Breadth-First Traversal (Level Order) BFT

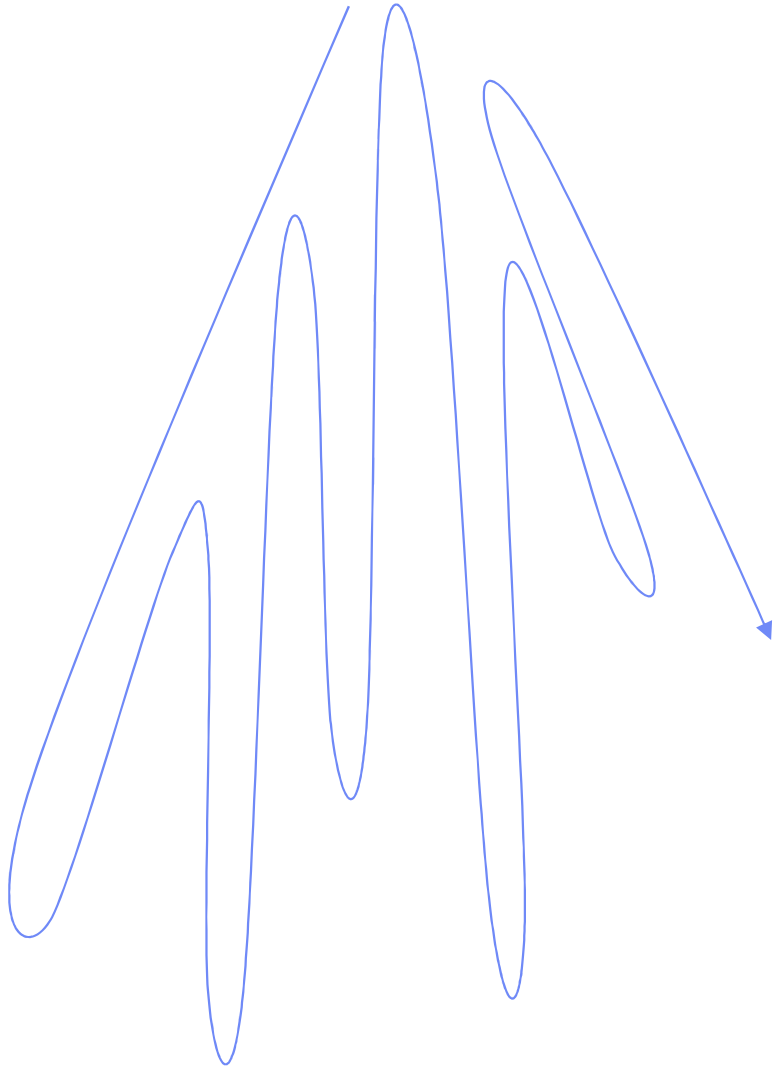
Breadth-first traversal is visiting each node starting from the root and moving down level by level, visiting nodes on each level from left to right.

F, D, J, B, E, G, K,
A, C, I, H



Depth-First Traversal DFT

Depth-First Traversal



Depth-first traversal proceeds as far as possible to the left (or right), then backs up until the first crossroad, goes one step to the right (or left), and again as far as possible to the left (or right). Repeat this process until all nodes are visited.

Depth-First Traversal

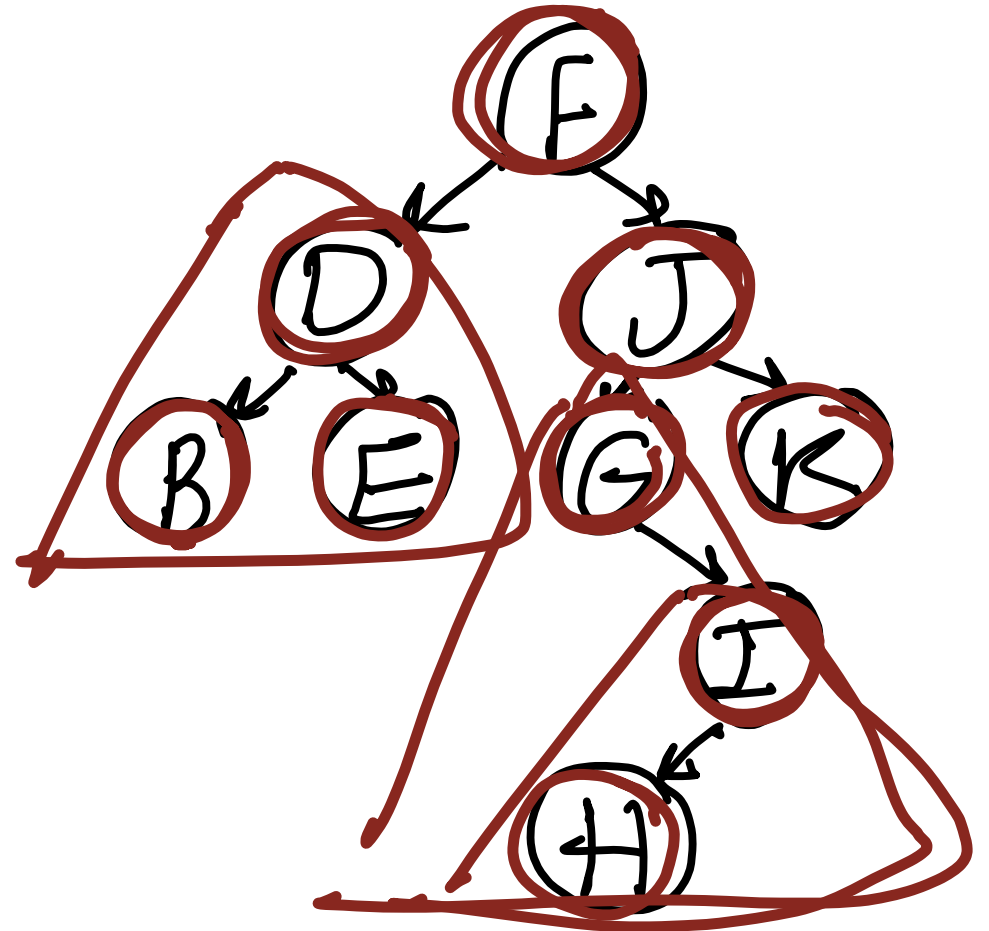
Preorder(root-left-right)

root

F

D, B, E, J

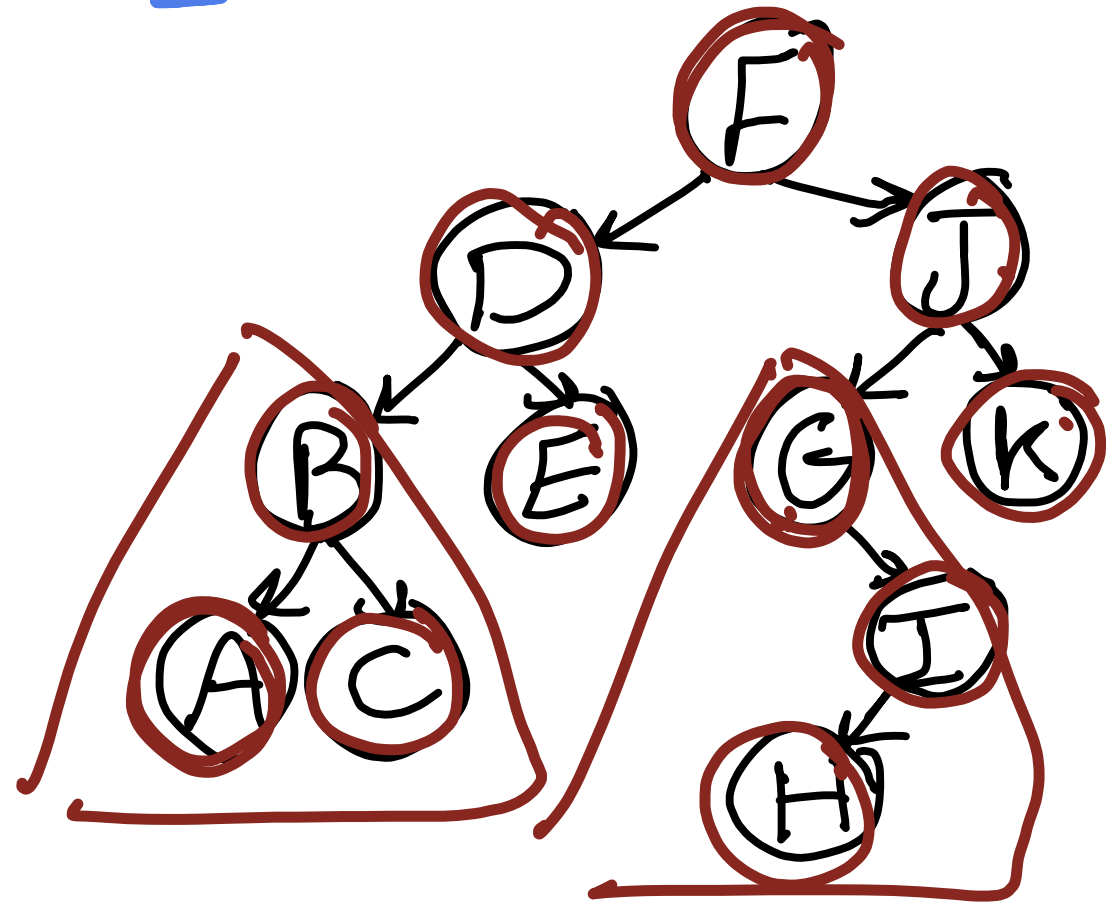
G, I, H, K



Depth-First Traversal

Inorder (left-root-right)

A, B, C, D, E
F, G, H, I
J, K

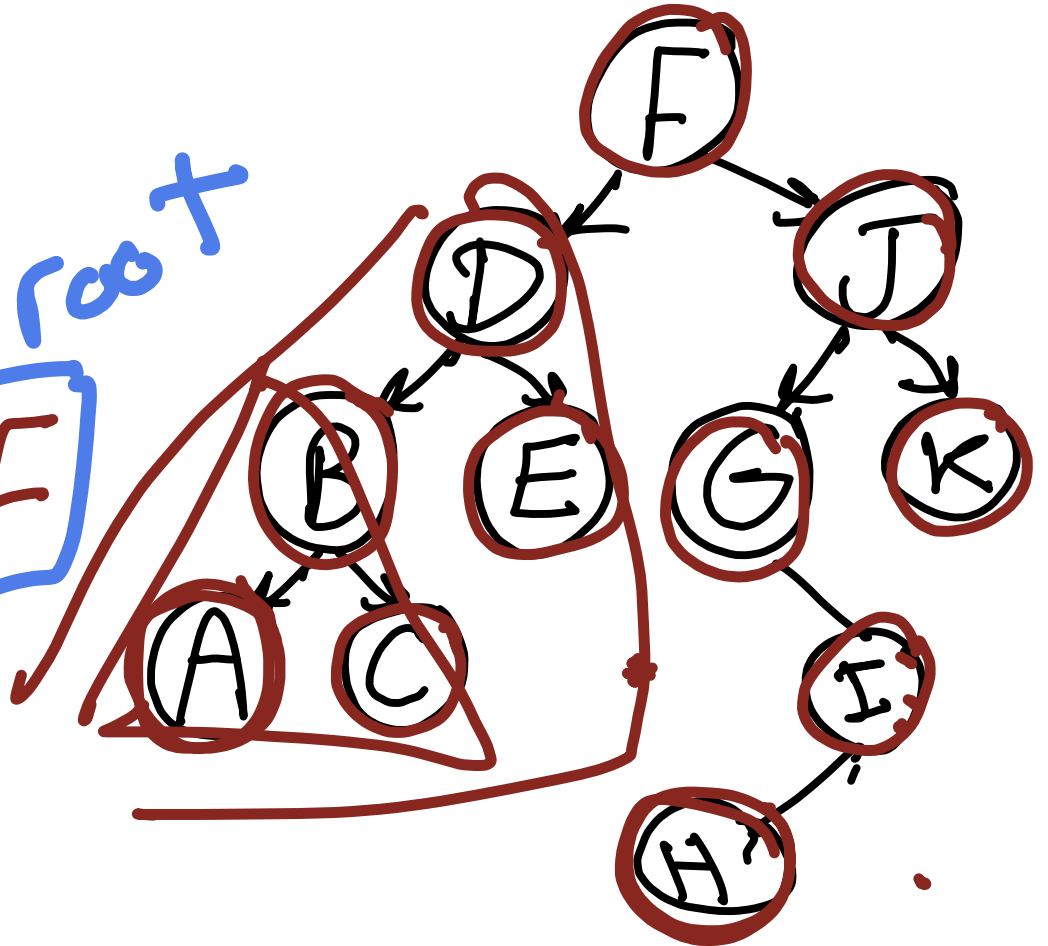


Depth-First Traversal

Postorder (left-right-root)

A, C, B, E,

H, I, G, K, J, F



Implementing of Binary tree traversal

1. Breadth First Traversal – Level order

Queue

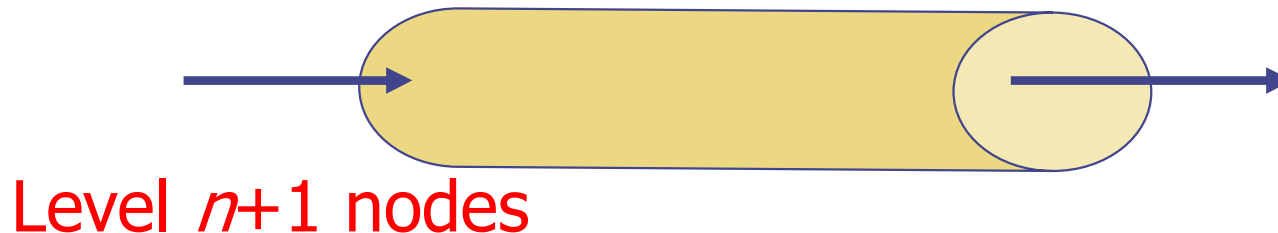
2. Depth First Traversal – Preorder \Inorder\Postorder

Stack

Breadth-First Traversal

Implementation of breadth-first traversal is quite straightforward when a **queue** is used. After a node is visited, its children are placed at the end of the queue, and the node at the beginning of the queue is visited.

Considering that for a node on level n , its children are on level $n+1$, by placing these children at the end of the queue, they are visited after all nodes from level n are visited.



Pseudo code

breadthFirst()

if (!isEmpty())

q = new Queue

q.enqueue(root)

while(!queue.isEmpty())

Node n = q.dequeue()

print n

if(n.left != null)

q.enqueue(n.left)

if (n.right != null)

q.enqueue(n.right)

Start at root (n), add n to queue

While queue is not empty

Dequeue head (n)

Output n

If n points at child

nodes, add to queue

Depth-First Traversal pseudocode

Preorder $\langle \text{root}, \text{left}, \text{right} \rangle$

Preorder(root)

{

if (root=NULL) return ;

→ print (root->data);

→ preorder(root->left);

→ preorder(root->right);

}

Depth-First Traversal pseudocode

Inorder < left - root - right

```
inorder(root)
```

```
{
```

```
    if (root=NULL) return ;
```

```
    → inorder(root->left);
```

```
    print (root->data);
```

```
    inorder(root->right);
```

```
}
```

Depth-First Traversal pseudocode

Postorder $\langle \text{left}, \text{right}, \text{root} \rangle$

```
Postorder(root)  
{  
    if (root=NULL) return ;  
    postorder(root->left);  
    postorder(root->right);  
    print (root->data);  
}
```